

# Projecto 01

## Laboratórios de Sistemas e Serviços

Mário Antunes

2026

### Projetos

Formem grupos de dois ou três alunos (excecionalmente, os projetos podem ser feitos individualmente) e selecionem **um** dos seguintes projetos. Todos os projetos serão alojados no **GitHub**, utilizando o [GitHub Classroom](#)). Consultem os detalhes [aqui](#).

O repositório deve conter todos os scripts relevantes, ficheiros de configuração e um README .md com instruções sobre como implementar o projeto. Deve também conter um relatório do projeto em formato PDF.

Este é um projeto de três semanas (data limite: 13/05/2026). Têm até ao final desta semana para notificar o vosso professor (por e-mail) sobre os elementos do grupo e o tema escolhido (a lista de temas pode ser encontrada [aqui](#)).

Não se esqueçam de contactar o vosso professor em caso de dúvidas. Instruções adicionais poderão ser adicionadas.

### Temas

#### 1. Site Estático de Alto Desempenho com Caching

- **Descrição:** Implementar um serviço web de alto desempenho utilizando o Docker Compose. Esta configuração deve incluir dois serviços: um servidor web (como o **Caddy** ou o **Apache httpd**) e uma cache de reverse proxy (como o **Squid**). O conteúdo estático do site (uma página complexa com vários estilos e imagens) deve ser servido a partir de um **volume** montado no contentor do servidor web. A cache deve ser configurada para ficar à frente do servidor web, e apenas a porta da cache deve estar exposta.
- **Tópicos Principais:** Docker Compose (multisserviço), Caddy/httpd, Squid, volumes, redes de contentores.

## 2. A Solução “Na Minha Máquina Funciona”: Um Dev Container

- **Descrição:** Criar um Dockerfile para uma linguagem de programação específica (ex., Python, C++ ou Node.js). Este Dockerfile deve instalar o compilador/interpretador e todas as bibliotecas necessárias. O projeto utilizará o Docker Compose e um **volume** para montar uma pasta de código local, permitindo compilar/executar o vosso código a partir do *interior* do contentor, garantindo um ambiente de build reproduzível.
- **Tópicos Principais:** Dockerfile, volumes, Docker Compose, gestão de pacotes (apt).

## 3. Backup Automatizado para o Nextcloud

- **Descrição:** Escrever um **script Bash** que crie um backup comprimido .tar.gz de um diretório específico. O script deverá então mover este arquivo para uma pasta local que esteja a ser monitorizada pelo **Nextcloud Desktop Client**. O objetivo é criar um sistema de backup totalmente automatizado onde os ficheiros locais são arquivados e, em seguida, sincronizados automaticamente para um servidor Nextcloud remoto.
- **Tópicos Principais:** Scripting em Bash (tar, date), cron, cliente Nextcloud.

## 4. Site de Anúncios da Disciplina com WordPress

- **Descrição:** Implementar uma instalação completa do WordPress utilizando o Docker Compose. Isto requer a orquestração de contentores wordpress e mysql (ou MariaDB). Devem utilizar **volumes** para persistência. O objetivo é configurar o site como um simples feed de anúncios para esta disciplina, criando pelo menos dois posts e personalizando o tema.
- **Tópicos Principais:** Docker Compose (multisserviço), WordPress, redes de contentores, volumes, variáveis de ambiente.

## 5. Duelo de Desempenho: VM vs. Contentor

- **Descrição:** Implementar um servidor web NGINX simples de duas formas: 1) dentro de uma **VM Debian** completa (utilizando Virtual-Box/QEMU) e 2) dentro de um **contentor Docker**. Devem depois escrever um relatório a comparar o tempo de arranque, a utilização de RAM em repouso (idle) e a ocupação de espaço em disco para ambos os métodos.
- **Tópicos Principais:** Virtualização (configuração de VM), Contentores (Docker), ferramentas de monitorização do sistema (top, df, time).

## 6. Implementação da Wiki da Disciplina

- **Descrição:** Utilizar o Docker Compose para implementar uma wiki totalmente funcional (ex., dokuwiki/dokuwiki ou linuxserver/bookstack) para servir como base de conhecimento para esta disciplina. O foco está na leitura correta da documentação da imagem, gestão de dados persistentes com **volumes** e configuração do serviço através de variáveis de ambiente. Devem preencher a wiki com pelo menos cinco páginas de conteúdo a partir dos materiais da disciplina.
- **Tópicos Principais:** Docker Compose, volumes, gestão de imagens de terceiros, variáveis de ambiente.

## Acesso ao GitHub Classroom

Aqui estão as instruções detalhadas para aceder ao GitHub Classroom.

### 1. Aderir ao Trabalho e Formar a Vossa Equipa

1. **Aceder ao link:** Cliquem [aqui](#)
2. **Encontrar o vosso nome:** Seleccionem o vosso nome a partir da lista de alunos. > **Não encontram o vosso nome?** Todos os nomes registados no PACO foram adicionados. Se o vosso estiver em falta, por favor contactem o [Prof. Mário Antunes](#).
3. **Criar uma equipa (APENAS UM elemento):** Apenas **uma** pessoa do vosso grupo deve criar uma equipa. Sigam esta estrutura de nomenclatura exata (os nmec devem estar ordenados): [nmec1]\_[nmec2]\_[nmec3]\_tema0[1-6]
  - (Exemplo: 132745\_133052\_tema02)
4. **Aderir à equipa (Todos os outros elementos):** Os restantes elementos do projeto devem encontrar e aderir à equipa criada no passo anterior.

---

### 2. Aceder à Organização e Repositório

1. **Aceitar o convite por e-mail:** Após aderirem a uma equipa, todos os elementos receberão um convite por e-mail para se juntarem à organização detiuaveiro no GitHub.
2. **Devem aceitar este convite** antes de poderem prosseguir.
3. **Atualizar a página:** Voltem à página do GitHub Classroom e atualizem-na (refresh).
4. **Verificar o acesso:** Deverão agora ver e ter acesso ao repositório de trabalho da vossa equipa.

---

### 3. Configurar uma Chave SSH para Acesso

Isto permitir-vos-á clonar (clone) e enviar (push) para o repositório a partir da vossa linha de comandos sem terem de introduzir a vossa palavra-passe de todas as vezes.

1. **Verificar se existe uma chave SSH:** Abram o vosso terminal e executem este comando:

```
cat ~/.ssh/id_ed25519.pub
```

2. **Gerar uma chave (se necessário):**

- Se virem uma chave (começando com `ssh-ed25519` . . .), copiem a linha inteira e saltem para o passo 3.
- Se virem um erro como “No such file or directory”, executem o seguinte comando para criar uma nova chave:

```
ssh-keygen -q -t ed25519 -N ''
```

- Depois de ser gerada, executem novamente `cat ~/.ssh/id_ed25519.pub` para visualizarem a vossa nova chave e copiem-na.

3. **Adicionar a chave à vossa conta GitHub:**

- Vão às **Settings** (Definições) do vosso GitHub.
- No menu do lado esquerdo, cliquem em **SSH and GPG keys**.
- Cliquem no botão **New SSH key**.
- Deem-lhe um **Title** (Título) (ex., “O meu Portátil da UA”).
- Colem a chave que copiaram no campo **Key** (Chave).
- Certifiquem-se de que o “Key type” (Tipo de chave) está definido como **Authentication Key**.
- Cliquem em **Add SSH key**.

4. **Autorizar a chave para SSO:**

- Depois de adicionarem a chave, encontrem-na na vossa lista na mesma página.
- Cliquem em **Configure SSO**.
- Seleccionem a organização **detiuaveiro**, preencham os vossos dados de início de sessão e concedam o acesso.